

Data Members & Member Functions

Designing object state and behavior - then seeing how Java resolves calls, passes data

CSC241 - Object-Oriented Programming

Fall 2026 | COMSATS University Islamabad, Lahore Campus

**Muhammad Shahid Bhatti | Assistant Professor | Department of Computer Science
| Office H-17**

Course repository: github.com/mshahidbhatti/CSC241-OOP

Student

studentId
name
completedCredits

addCredits(int)
remainingCredits(int)
summary()

Today: fields hold state; methods define behavior. We will compile, predict, run, and explain at each checkpoint.

Bridge from Lecture 3

Classes and objects gave us the container. Lecture 4 focuses on the members inside that container.

Lecture 3

class, object, reference, identity, state

Today

instance variables, methods, parameters, returns

Then

access control & encapsulation in Lecture 5

Continuity rule: we intentionally use the implicit no-argument constructor and package-visible fields today; constructors and access modifiers come later in the CDF.

Lecture 4 scope from the CDF

This lecture stays tightly within the official topic list.

- Instance variables and methods
- Method overloading
- Java naming conventions
- Method signatures and the role of parameter types in overload resolution
- Primitive argument passing vs. passing object references by value in Java
- Return types

Teaching mode

Predict → Code → Compile → Run → Explain

Short live-coding cycles are used throughout so students

Not yet: encapsulation/access modifiers, constructors, static-member semantics, inheritance, generics. The familiar static main/helper syntax may appear only as boilerplate.

Learning outcomes

By the end of the lecture, students should be able to:

- Declare meaningful instance variables and explain why each object owns its own instance state.
- Write instance methods with parameters, local variables, and appropriate return types.
- Apply standard Java naming conventions to classes, fields, methods, parameters, and constants.
- Explain a Java method signature and distinguish valid from invalid overloads.
- Predict which overloaded method the compiler selects for straightforward calls.
- Explain Java pass-by-value for primitives and for object references using diagrams and code.

Running example: students in our class

We will use familiar class data to make object state concrete.

```
class Student {  
    String studentId;  
    String name;  
    int completedCredits;  
}
```

s1

studentId = "CIIT/SP26-BAI-005/LHR"
name = "Abeer Amina"
completedCredits = 15

s2

studentId = "CIIT/SP26-BAI-010/LHR"
name = "Ali Ishtiaq"
completedCredits = 18

One class definition; many independent objects. The field names are shared by the type, but each object stores its own values.

Instance variables: state stored inside each object

A field declared in a class (without static) is an instance variable.

```
class Student {  
    String studentId;  
    String name;  
    int completedCredits;  
}
```

State answers: “What does this object know?”

studentId → identity within our domain
name → descriptive state
completedCredits → changing academic state

Design check

Keep a field when the value belongs to the object across its lifetime

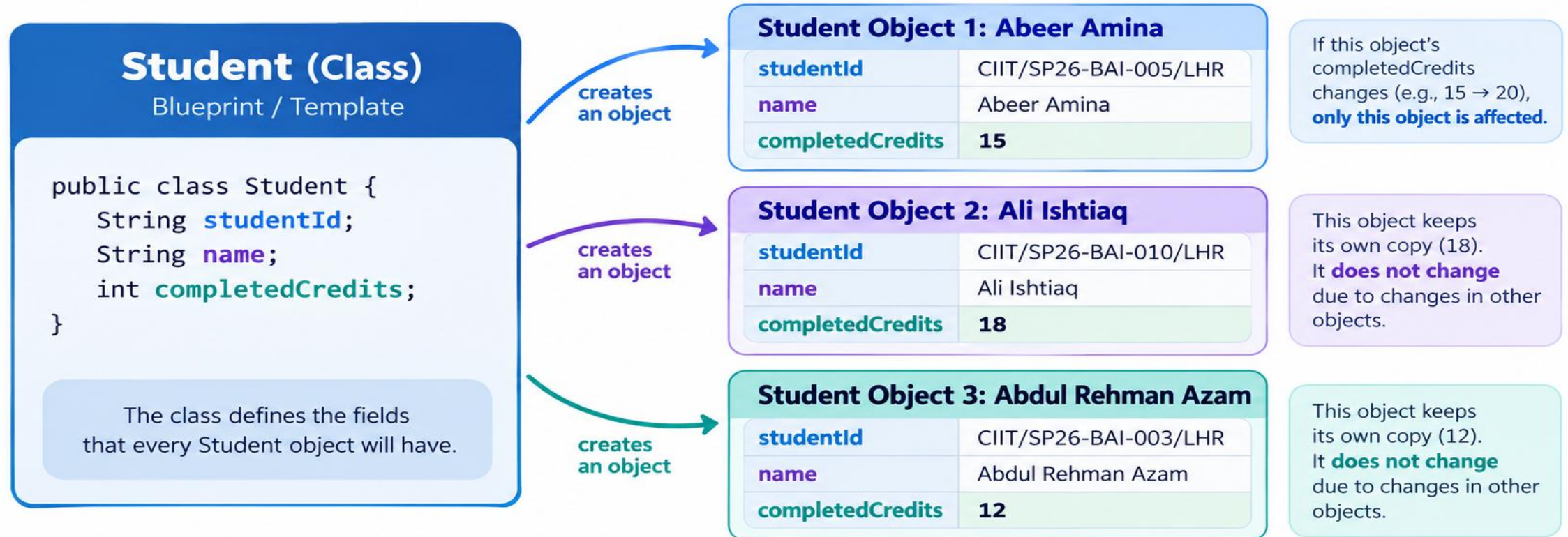
Avoid temporary calculations as fields when a local variable can do the job

A field is part of object state; a local variable belongs to one method execution. We will contrast them shortly.

Instance means “each object gets its own copy”

The class specifies the fields; each object stores independent field values.

A class defines the structure (fields). Each object has its own copy of those fields.



Each object has its own independent copy of instance fields.

Changing one object's completedCredits does not affect the others.

Field vs. local variable

Both hold data, but they live in different scopes and for different durations.

```
class Student {  
    int completedCredits;    // field  
  
    int remainingCredits(int degreeCredits) {  
        int remaining = degreeCredits - completedCredits; //  
        return remaining;  
    }  
}
```

Instance field

Belongs to the object
Persists as part of object state
Accessible to instance methods of the object

Local variable

Created for one method call
Used for temporary work
Disappears when that call finishes

Ask the class: should “remaining” be stored as a field, or can it be computed when needed?

Member functions in Java are methods

Methods define what an object can do with its state.

```
class Student {  
    int completedCredits;  
  
    void addCredits(int credits) {  
        completedCredits = completedCredits + credits;  
    }  
  
    int remainingCredits(int degreeCredits) {  
        return degreeCredits - completedCredits;  
    }  
}
```

Behavior 1

addCredits(3)
changes the object's state

Behavior 2

remainingCredits(130)
computes and returns a result

Key idea

A method may read state, modify state, compute a result, or combine these responses

Anatomy of a Java method

Read a method declaration from left to right.

```
int remainingCredits(int degreeCredits) {  
    int remaining = degreeCredits - completedCredits;  
    return remaining;  
}
```

return type

int

method name

remainingCredits

parameter

int degreeCredits

local variable

remaining

return value

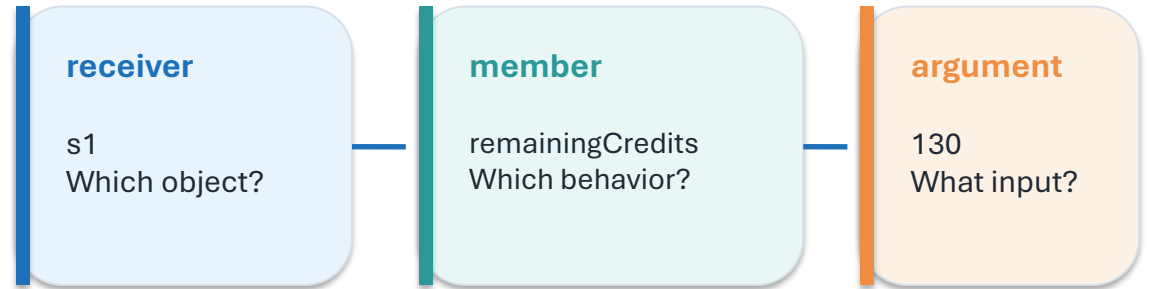
remaining

Method declaration = contract with the caller: what to call, what input to supply, and what result (if any) comes back.

Calling an instance method: the dot selects the receiver

The method executes for the object on the left side of the dot.

```
Student s1 = new Student();  
s1.name = "Abeer Amina";  
s1.completedCredits = 15;  
  
s1.addCredits(3);  
int left = s1.remainingCredits(130);
```



After addCredits(3): completedCredits = 18
remainingCredits(130) returns 112

Parameters vs. arguments

A parameter is declared by the method; an argument is supplied by the caller.

```
void addCredits(int credits) {  
    completedCredits += credits;  
}
```

Parameter

int credits

A named input slot in the method declaration

Argument

3

The actual value supplied at this call site

```
s1.addCredits(3);
```

At call time Java copies the argument value into the parameter. This single rule explains both primitive and object-reference passing later.

Return types tell the caller what comes back

A method can return no value, a primitive value, or a reference value.

void

```
void printSummary()
```

Performs work but returns no value to the caller.

primitive

```
int remainingCredits(int total)
```

Returns a directly stored primitive value.

reference type

```
String summary()
```

Returns a reference to a String object.

For a non-void method, every reachable execution path must provide a compatible return value.

Live coding 1 - add fields, methods, and return values

Build from the Lecture 3 Student class, compile after each small change.

LIVE CODING

```
// Student.java
class Student {
    String studentId;
    String name;
    int completedCredits;

    void addCredits(int credits) {
        completedCredits += credits;
    }

    int remainingCredits(int totalCredits) {
        return totalCredits - completedCredits;
    }
}
```

```
// Demo.java
class Demo {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Abeer Amina";
        s1.completedCredits = 15;

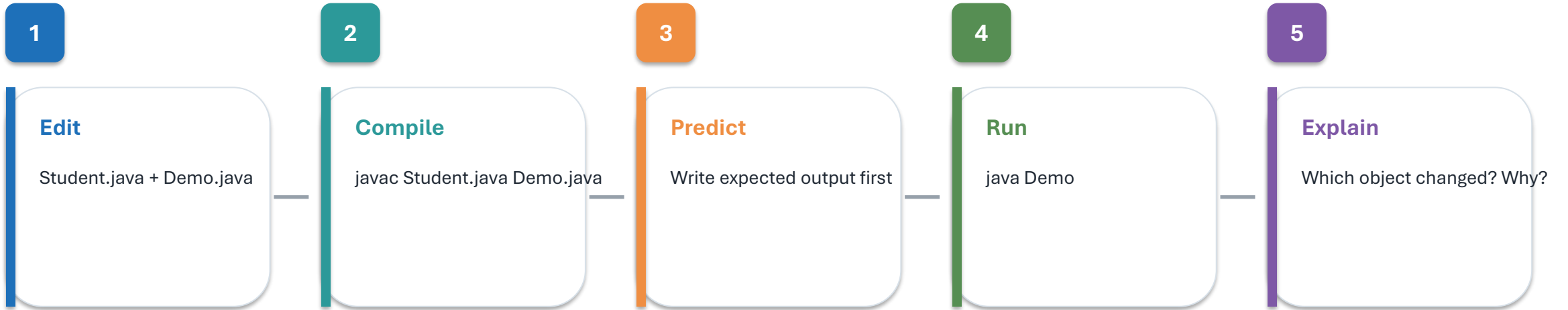
        s1.addCredits(3);
        System.out.println(s1.remainingCredits(130));
    }
}
```

Live coding folder: 00_Course_Master\01_Lectures\Week02\L04\LiveCoding

Predict before running: what prints?

Compile-run-explain cycle

Do not hide compilation from students - let compiler feedback drive the discussion.



Suggested deliberate error: call `s1.addCredits();` with no argument. Ask students to interpret the compiler message before fixing it.

Java naming conventions

Readable names are part of the class interface students and teammates must understand.

Class	UpperCamelCase	Student, CourseSection
Method	lowerCamelCase	addCredits(), remainingCredits()
Field / variable	lowerCamelCase	studentId, completedCredits
Constant	UPPER_SNAKE_CASE	MAX_CREDITS

Prefer intent-revealing names: calculateRemainingCredits() communicates more than calc() or doWork().

Constant naming is shown only as a convention preview; static/final semantics come later

Names are part of your class interface

A method name should tell the caller what behavior to expect.

```
void upd(int x) {  
    cc += x;  
}
```

Hard to read

```
void addCredits(int credits) {  
    completedCredits += credits;  
}
```

Intent is clear

Classroom question: what assumptions can a caller make from the method name alone?

What exactly is a method signature in Java?

For overloading, the compiler distinguishes methods by name + parameter types.

```
void enroll(String courseCode, int section) { ... }
```

Signature

enroll(String, int)

Included

method name
number of parameters
parameter types
parameter order

Not enough to distinguish overloads

return type
parameter names
local variables

Important precision: in Java, the method signature is the method name plus the parameter types (and type parameters, if any). The return type is not part of the signature.

The compiler matches a call to a method declaration

Argument count and argument types must match an accessible declaration.

```
void addCredits(int credits) { ... }
```

```
s1.addCredits(3);
```

✓

one int argument

```
s1.addCredits();
```

✗

missing argument

```
s1.addCredits(3, 4);
```

✗

too many arguments

```
s1.addCredits("3");
```

✗

String is not int

Method overloading: same operation name, different parameter lists

Overloading lets one conceptual operation accept different forms of input.

```
void enroll(String courseCode) { ... }  
void enroll(String courseCode, int section) { ... }  
void enroll(int courseNumericId) { ... }
```

Same method name

enroll

Different parameter lists

Overloading is compile-time polymorphism; detailed polymorphism comes later in the course.

Valid overload #1 - different number of parameters

The parameter count alone can make signatures distinct.

```
void recordScore(int score) { ... }  
void recordScore(int score, int weight) { ... }
```

Call A

recordScore(85);
→ one-parameter overload

Call B

recordScore(85, 20);
→ two-parameter overload

Ask students to state the signatures aloud: recordScore(int) and recordScore(int, int).

Valid overload #2 - different parameter types

Same count, different types: still distinct signatures.

```
void findStudent(int numericId) { ... }  
void findStudent(String studentId) { ... }
```

findStudent(5)

argument type = int
compiler selects findStudent(int)

findStudent("CIIT/SP26-BAI-005/LHR")

argument type = String
compiler selects findStudent(String)

The compiler decides from the call-site argument types - before the program runs.

Valid overload #3 - same types, different order

Parameter order is part of the signature.

```
void assign(String courseCode, int section) { ... }  
void assign(int section, String courseCode) { ... }
```

Call 1

assign("CSC241", 2)
→ (String, int)

Call 2

assign(2, "CSC241")
→ (int, String)

Design caution: overloads that differ only by order can be hard to read. Valid Java does not automatically mean good API design.

Invalid overload - return type alone is not enough

Java cannot overload two methods that have the same name and parameter types.

COMPILER ERROR

```
int calculateLeft(int credits) { ... }  
double calculateLeft(int credits) { ... }
```

Why rejected?

Both declarations have the same signature:

calculateLeft(int)

The return type does not distinguish overloads.

Classroom question: how would the compiler choose a method for calculateLeft(10) before seeing what you do with the result?

Overload resolution: start with the argument list

At an introductory level, count arguments, inspect their types, then select the compatible signature.

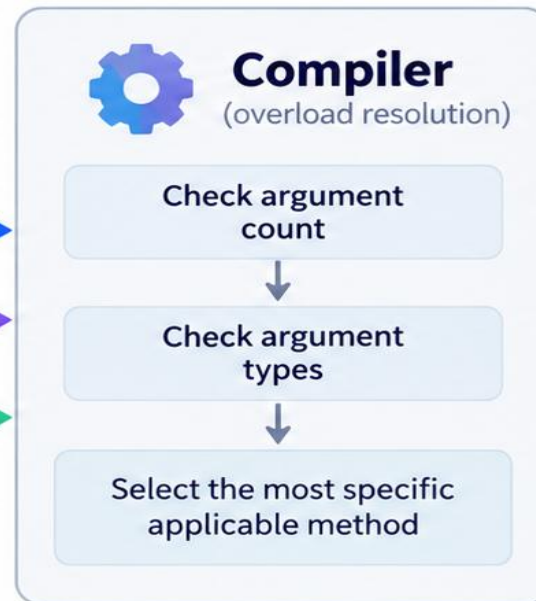
Same method name, different parameter lists. The compiler selects the best matching method at **compile time**.

Method Calls (at compile time)

1 enroll("CSC241")

2 enroll("CSC241", 2)

3 enroll(241)



Candidate Methods (overloaded)

void enroll(String course)

void enroll(String course, int section)

void enroll(int courseNumber)



The method is chosen at compile time, not at runtime.

The compiler uses the argument count and types to find the best matching overloaded method.

When there is no exact match: compatible numeric conversion

Java may use a legal widening primitive conversion during overload resolution.

```
void show(long value) { System.out.println("long"); }  
void show(double value) { System.out.println("double"); }  
  
show(10);
```

What happens?

The literal 10 has type int.
No show(int) exists.
int can widen to long and double.
long is the more specific applicable choice here.

Keep this as a guided example only. More complex overload resolution rules are not the goal of this lecture.

Live coding 2 - overload a method and let the compiler choose

Add one overload at a time, compile, then predict each call.

LIVE CODING

```
class Student {  
    String name;  
  
    void printLabel() {  
        System.out.println(name);  
    }  
  
    void printLabel(String prefix) {  
        System.out.println(prefix + ": " + name);  
    }  
}
```

```
Student s = new Student();  
s.name = "Ali Ishtiaq";  
  
s.printLabel();  
s.printLabel("Student");  
  
// Try next:  
// s.printLabel(3);
```

Student task

Before compiling each call:

1. write the expected signature
2. predict compile / error
3. justify using argument count + type

Java has one parameter-passing rule: pass by value

What gets copied depends on the value stored in the argument expression.

The value of the argument is **copied** into the parameter.

1. Primitive Argument (int)

A **copy** of the **value** is passed. Changes to the parameter do not affect the caller's variable.

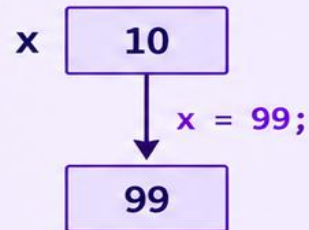
Caller

```
int n = 10;
```



Method

```
void changeNumber(int x) {  
    x = 99;  
}
```



Result

- **x** is a copy of **n**'s value.
- **x** changes to **99** inside the method.
- **n** in the caller remains **10**.

2. Object Reference Argument (Student)

A **copy** of the reference value is passed. Both variables refer to the same object, so changes to the object are seen by the caller.

Caller

```
Student s = new Student();  
s.completedCredits = 10;
```

Stack (caller)

s

Heap

Student object
completedCredits = 10

Method

```
void changeStudent(Student st) {  
    st.completedCredits = 99;  
}
```

Stack (method)

st

Copy reference value into parameter st



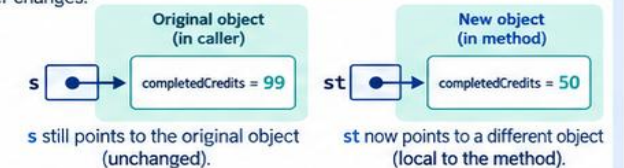
Result

- **st** is a copy of **s**'s reference (both point to the same object).
- **st.completedCredits = 99;** changes the object.
- **s.completedCredits** in the caller is now **99**.

Reassigning the parameter (does not affect the caller)

If **st** is assigned to a new object, only the local parameter changes. The caller's **s** still points to the original object.

```
void changeStudent(Student st) {  
    st = new Student(); // st now points to a different object.  
    st.completedCredits = 50;  
}
```



Primitive argument: the method changes only its local copy

Changing the parameter does not change the caller's primitive variable.

```
static void addBonus(int credits) {  
    credits = credits + 3;  
}
```

```
int c = 15;  
addBonus(c);  
System.out.println(c);
```

Caller before

c = 15

Parameter copy

credits = 15
then 18

After method returns: c is still 15

Predict before running. Ask: which storage location did the assignment `credits = credits + 3` actually update?

static appears only as helper/main boilerplate here; static-member semantics are covered later.

Object argument: the reference value is copied

Two reference variables can temporarily point to the same object.

```
static void addCredits(Student st) {  
    st.completedCredits += 3;  
}
```

```
Student s1 = new Student();  
s1.completedCredits = 15;  
addCredits(s1);
```

caller variable

s1
(reference value)

parameter

st
(copy of same reference)

Student object

completedCredits = 15 → 18

Both references reach the same object, so mutating st.completedCredits is visible through s1 after the call.

static appears only as helper/main boilerplate here; static-member semantics are covered later.

Reassigning the copied reference does NOT rebind the caller

A parameter can be redirected locally, but the caller variable keeps its original reference.

```
static void replace(Student st) {  
    st = new Student();  
    st.name = "Temporary";  
}
```

```
Student s1 = new Student();  
s1.name = "Abeer Amina";  
replace(s1);  
System.out.println(s1.name);
```

Before call

s1 → object #1
name = Abeer Amina

Inside replace()

st initially → object #1
then st → object #2
name = Temporary

Caller still prints: Abeer Amina

Mutation through the copied reference can affect the shared object; reassigning the parameter only changes the local copy of the reference.

static appears only as helper/main boilerplate here; static-member semantics are covered later.

Live coding 3 - prove pass-by-value with two experiments

Run primitive and object-reference cases side by side.

LIVE CODING

```
static void changeNumber(int x) {  
    x = 99;  
}
```

```
int n = 10;  
changeNumber(n);  
System.out.println(n);
```

```
static void changeStudent(Student st) {  
    st.completedCredits = 99;  
}
```

```
Student s = new Student();  
s.completedCredits = 10;  
changeStudent(s);  
System.out.println(s.completedCredits);
```

Predict first

Left output = ?
Right output = ?
Explain both using “copied value.”

static appears only as helper/main boilerplate here; static-member semantics are covered later.

Integrated live-coding challenge

Students modify a small class, predict the calls, then compile and run.

LIVE CODING

```
class Student {  
    String studentId;  
    String name;  
    int completedCredits;  
  
    void addCredits(int c) { completedCredits += c; }  
    void addCredits(int c, int bonus) {  
        completedCredits += c + bonus;  
    }  
    int remainingCredits(int total) {  
        return total - completedCredits;  
    }  
}
```

1. Write the signatures
2. Predict completedCredits
3. Predict printed result
4. Add an invalid overload and explain the compiler error

```
Student s = new Student();  
s.studentId = "CIIT/SP26-BAI-003/LHR";  
s.name = "Abdul Rehman Azam";  
s.completedCredits = 12;  
  
s.addCredits(3);  
s.addCredits(3, 1);  
System.out.println(s.remainingCredits(130));
```

Student tasks

1. Write both addCredits(...) signatures.
2. Trace completedCredits after each call.
3. Predict the final printed value.
4. Add one invalid overload and explain the compiler message.
5. Explain which values are copied at each method call.

Compiler clinic - can Java resolve these calls?

Decide compile/error before running anything.

```
void enroll(String code) { ... }  
void enroll(String code, int section) { ... }  
void enroll(int numericCode) { ... }
```

enroll("CSC241");

COMPILES

enroll(String)

enroll("CSC241", 2);

COMPILES

enroll(String, int)

enroll(241);

COMPILES

enroll(int)

enroll();

ERROR

no matching signature

enroll("241", "2");

ERROR

second argument is String

Lecture 4 recap + exit ticket

The CDF concepts now fit together as one call flow.

Fields

store per-object state

Methods

define object behavior

Signature

name + parameter types

Overloading

same name, distinct signatures

Arguments

their values are copied

Exit ticket - answer without running code

- 1. What is the signature of `enroll(String, int)`?
- 2. Can methods differ only by return type? Why?
- 3. After passing an `int` to a method, can the method reassign the caller's `int` variable?
- 4. Why can mutating an object through a parameter be visible to the caller?
- 5. What changes next lecture when fields become private?

Next: Lecture 5 - Access Modifiers & Encapsulation.

Bridge: next lecture we protect this state with access control, getters/setters, validation, and class invariants.